

GUION DOCENTE — CLASE 2

PostgreSQL + SQLAlchemy ORM + Modelado + Migraciones

Duración estimada: 3 horas

Modalidad: práctica guiada

Proyecto: Dashboard SaaS Café Andino

Nivel: intermedio — Programación Web + Bases de Datos

1. Objetivo general de la clase

Conectar el proyecto Flask con una base de datos PostgreSQL, definir modelos mediante SQLAlchemy ORM, crear tablas con migraciones y realizar las primeras operaciones de inserción y consulta desde Python.

2. Resultado final de la clase

Al finalizar la sesión, el estudiante tendrá:

- PostgreSQL conectado al proyecto Flask.
 - Modelos ORM definidos.
 - Tablas creadas mediante migraciones.
 - Usuario inicial registrado.
 - Primeras consultas ORM ejecutadas.
 - Dashboard mostrando al menos un dato consultado desde la base de datos.
-

3. Idea central para explicar al estudiante

“En la Clase 1 construimos la puerta del sistema: login, rutas y estructura. Hoy construiremos el corazón del sistema: la base de datos y los modelos ORM.”

4. Estructura de la clase

Bloque	Tema	Tiempo
1	Repaso Clase 1 y objetivo de Clase 2	10 min
2	¿Qué es ORM y por qué usarlo?	20 min

Bloque	Tema	Tiempo
3	Configuración de PostgreSQL	25 min
4	Inicialización de SQLAlchemy y Flask-Migrate	25 min
5	Modelo Usuario	30 min
6	Modelo Venta	35 min
7	Migraciones	35 min
8	Inserción y consulta de datos	25 min
9	Integración mínima con dashboard	20 min
10	Práctica final y cierre	15 min

5. Archivos que se abrirán en orden

1. `run.py`
2. `app/__init__.py`
3. `app/config.py`
4. `app/models.py`
5. `app/auth/routes.py`
6. `app/dashboard/routes.py`
7. `migrations/`
8. `.env` o `.env.example`
9. `requirements.txt`

6. Archivos que NO se trabajan todavía

No profundizar aún en:

- `app/data/services.py`
 - `app/data/routes.py`
 - `dashboard.js`
 - `datatables.js`
 - gráficos Chart.js
 - importación CSV
 - dashboard completo
-

BLOQUE 1 — Repaso de Clase 1

Tiempo: 10 min

Qué mostrar

Ejecutar el proyecto y mostrar:

- login
- dashboard protegido
- logout

Qué decir

“Ya tenemos una aplicación protegida. Ahora necesitamos que los datos no estén en memoria ni quemados en el código, sino almacenados en PostgreSQL.”

Pregunta de activación

- ¿Dónde estaban los usuarios en la Clase 1?
- ¿Por qué no es correcto dejar usuarios quemados en el código?

BLOQUE 2 — ¿Qué es ORM?

Tiempo: 20 min

Concepto principal

ORM significa Object Relational Mapping. Permite trabajar con tablas de base de datos como si fueran clases de Python.

Explicación docente

Base de datos	Python ORM
Tabla	Clase
Columna	Atributo
Fila	Objeto
INSERT	<code>db.session.add()</code>
SELECT	<code>.query</code>

`COMMIT``db.session.commit()`

`## Qué decir`

> “Con ORM no dejamos de usar bases de datos; usamos una forma más ordenada de conectarlas con el código.”

`## Ejemplo conceptual`

```
usuario = Usuario(username="admin")
db.session.add(usuario)
db.session.commit()
```

`## Idea clave`

El ORM no reemplaza el conocimiento de SQL. Lo complementa y lo integra con programación web.

BLOQUE 3 — Configuración de PostgreSQL

Tiempo: 25 min

Archivo principal

`app/config.py`

Qué explicar

- conexión a PostgreSQL
- variables de entorno
- URI de conexión
- seguridad de credenciales

Ejemplo de configuración

```
import os
from dotenv import load_dotenv
load_dotenv()

class Config:
    SECRET_KEY = os.getenv("SECRET_KEY", "clave-desarrollo")
```

```
SQLALCHEMY_DATABASE_URI = os.getenv("DATABASE_URL")
SQLALCHEMY_TRACK_MODIFICATIONS = False
```

Ejemplo de `.env`

```
SECRET_KEY=clave-secreta-dashboard
DATABASE_URL=postgresql://postgres:admin@localhost:5432/cafe_andino_db
```

Qué decir

“Nunca debemos escribir credenciales reales directamente dentro del código fuente.”

Práctica

Cada estudiante debe verificar:

- PostgreSQL activo.
- Base de datos creada.
- Usuario y contraseña correctos.
- URI configurada en `.env`.

BLOQUE 4 — Inicialización de SQLAlchemy y Flask-Migrate

Tiempo: 25 min

Archivo

`app/__init__.py`

Código base a explicar

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate
from flask_login import LoginManager
from app.config import Config

db = SQLAlchemy()
migrate = Migrate()
```

```
login_manager = LoginManager()

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)
    db.init_app(app)
    migrate.init_app(app, db)
    login_manager.init_app(app)
    login_manager.login_view = "auth.login"
    from app.auth.routes import auth_bp
    from app.dashboard.routes import dashboard_bp
    app.register_blueprint(auth_bp)
    app.register_blueprint(dashboard_bp)
    return app
```

Qué explicar línea por línea

- `db = SQLAlchemy()` crea el objeto ORM.
- `migrate = Migrate()` habilita migraciones.
- `login_manager` administra sesiones.
- `db.init_app(app)` conecta ORM con Flask.
- `migrate.init_app(app, db)` conecta migraciones con la app y la base de datos.

Mensaje docente

“Aquí se conectan las tres piezas principales: Flask, PostgreSQL y SQLAlchemy.”

BLOQUE 5 — Modelo Usuario

Tiempo: 30 min

Archivo

`app/models.py`

Código de referencia

```
from datetime import datetime
from flask_login import UserMixin
from werkzeug.security import generate_password_hash, check_password_hash
from app import db, login_manager
```

```

class Usuario(UserMixin, db.Model):
    __tablename__ = "usuarios"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    nombre_completo = db.Column(db.String(120), nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
    rol = db.Column(db.String(30), default="analista")
    estado = db.Column(db.Boolean, default=True)
    fecha_creacion = db.Column(db.DateTime, default=datetime.utcnow)
    ultimo_acceso = db.Column(db.DateTime)

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

@login_manager.user_loader
def load_user(user_id):
    return Usuario.query.get(int(user_id))

```

Qué explicar

- `UserMixin` permite integración con Flask-Login.
- `primary_key=True` define clave primaria.
- `unique=True` evita duplicados.
- `nullable=False` hace obligatorio el dato.
- `set_password()` guarda contraseña segura.
- `check_password()` valida contraseña.
- `user_loader` permite recuperar usuario desde la sesión.

Pregunta para estudiantes

- ¿Por qué no se guarda la contraseña en texto plano?

BLOQUE 6 — Modelo Venta

Tiempo: 35 min

Archivo

app/models.py

Código de referencia

```
class Venta(db.Model):
    __tablename__ = "ventas"
    id = db.Column(db.Integer, primary_key=True)
    id_venta_csv = db.Column(db.String(50), unique=True, nullable=False)
    fecha_venta = db.Column(db.Date)
    hora_venta = db.Column(db.String(20))
    canal_venta = db.Column(db.String(50))
    sucursal = db.Column(db.String(100))
    region_sucursal = db.Column(db.String(100))
    id_cliente = db.Column(db.String(50))
    nombre_cliente = db.Column(db.String(150))
    tipo_cliente = db.Column(db.String(80))
    segmento_cliente = db.Column(db.String(80))
    ciudad_cliente = db.Column(db.String(100))
    zona_cliente = db.Column(db.String(100))
    nombre_producto = db.Column(db.String(150))
    categoria_producto = db.Column(db.String(100))
    linea_producto = db.Column(db.String(100))
    presentacion = db.Column(db.String(100))
    origen_cafe = db.Column(db.String(100))
    tipo_tueste = db.Column(db.String(80))
    precio_unitario = db.Column(db.Float)
    cantidad = db.Column(db.Integer)
    descuento_porcentaje = db.Column(db.Float)
    total_bruto = db.Column(db.Float)
    total_descuento = db.Column(db.Float)
    total_netto = db.Column(db.Float)
    fecha_carga = db.Column(db.DateTime, default=datetime.utcnow)
```

Qué explicar

- Este modelo representa el CSV que luego se importará.
- Cada columna del CSV se convierte en un atributo del modelo.
- `id_venta_csv` evita cargar ventas duplicadas.
- Los campos numéricos permitirán agregaciones.

Relación con la Clase 3

"Hoy solo modelamos la tabla. En la próxima clase cargaremos el CSV y llenaremos esta tabla."

BLOQUE 7 — Migraciones

Tiempo: 35 min

Qué explicar

Una migración es el historial de cambios estructurales de la base de datos. Permite crear y modificar tablas sin hacerlo manualmente.

Comandos

```
flask db init
flask db migrate -m "creacion inicial de tablas"
flask db upgrade
```

Explicación de cada comando

- `flask db init`: crea carpeta de migraciones.
- `flask db migrate`: detecta cambios en modelos y genera archivo de migración.
- `flask db upgrade`: aplica la migración en PostgreSQL.

Errores frecuentes

Error: no reconoce flask db

Solución: verificar que Flask-Migrate esté instalado y registrado.

```
pip install Flask-Migrate
```

Error: no conecta a PostgreSQL

Solución: revisar `DATABASE_URL`, usuario, contraseña, puerto y nombre de base.

Error: relación ya existe

Solución: revisar si ya se ejecutó migración previa o limpiar base en entorno de práctica.

Práctica

Cada estudiante debe: 1. crear base en PostgreSQL. 2. configurar `.env`. 3. ejecutar migraciones. 4. verificar tablas en DBeaver o pgAdmin.

BLOQUE 8 — Inserción y consulta de datos

Tiempo: 25 min

Objetivo

Insertar el primer usuario real en la base de datos y consultarlo desde Python.

Opción recomendada: usar Flask shell

```
flask shell
```

Código en shell

```
from app import db
from app.models import Usuario

u = Usuario(
    username="admin",
    nombre_completo="Administrador del Sistema",
    email="admin@cafeandino.com",
    rol="admin",
    estado=True
)
u.set_password("admin123")
db.session.add(u)
db.session.commit()
```

Consulta

```
Usuario.query.all()
Usuario.query.filter_by(username="admin").first()
```

Qué explicar

- `db.session.add()` prepara el objeto para guardar.
- `db.session.commit()` confirma la operación.
- `.query.all()` devuelve todos los registros.
- `.first()` devuelve el primer resultado.

Pregunta de cierre del bloque

- ¿Qué pasaría si hacemos `add()` pero no hacemos `commit()`?

BLOQUE 9 — Integración mínima con login y dashboard

Tiempo: 20 min

Archivo

`app/auth/routes.py`

Ajuste conceptual

En Clase 1 el usuario podía estar hardcodedo. Ahora el login debe buscar al usuario en PostgreSQL.

Código base de login con ORM

```
@auth_bp.route("/login", methods=["GET", "POST"])
def login():
    form = LoginForm()
    if form.validate_on_submit():
        usuario = Usuario.query.filter_by(username=form.username.data).first()
        if usuario and usuario.check_password(form.password.data) and
usuario.estado:
            login_user(usuario)
            usuario.ultimo_acceso = datetime.utcnow()
            db.session.commit()
            return redirect(url_for("dashboard.index"))
        flash("Usuario o contraseña incorrectos", "danger")
    return render_template("login.html", form=form)
```

Qué explicar

- Ya no validamos contra variables internas.
- Buscamos al usuario en la tabla `usuarios`.
- Validamos password con hash.
- Actualizamos último acceso.

Archivo

app/dashboard/routes.py

Ejemplo mínimo

```
@dashboard_bp.route("/")
@login_required
def index():
    total_usuarios = Usuario.query.count()
    return render_template("dashboard.html", total_usuarios=total_usuarios)
```

Qué decir

"Este es el primer dato real del sistema mostrado en la interfaz."

BLOQUE 10 — Práctica final y cierre

Tiempo: 15 min

Actividad final obligatoria

Cada estudiante debe lograr: 1. Conectar Flask con PostgreSQL. 2. Crear modelos `Usuario` y `Venta`. 3. Ejecutar migraciones. 4. Insertar usuario administrador. 5. Iniciar sesión con usuario de base de datos. 6. Mostrar un dato consultado desde PostgreSQL en el dashboard.

Evidencias solicitadas

- Captura de tablas en PostgreSQL.
 - Captura de usuario insertado.
 - Captura del login exitoso.
 - Captura del dashboard mostrando un dato consultado.
-

7. Preguntas de evaluación rápida

- ¿Qué es un ORM?
- ¿Qué representa una clase en SQLAlchemy?
- ¿Qué hace `db.session.commit()`?
- ¿Para qué sirven las migraciones?
- ¿Por qué no se guarda la contraseña en texto plano?

- ¿Qué hace `user_loader`?
 - ¿Qué diferencia existe entre `.all()` y `.first()`?
-

8. Errores frecuentes y solución rápida

1. Error de conexión a PostgreSQL

Revisar `DATABASE_URL`, usuario, contraseña, host, puerto y nombre de base.

2. No reconoce `flask db`

Instalar y registrar Flask-Migrate.

3. No encuentra el modelo

Verificar imports en `models.py` y en `__init__.py`.

4. Login no funciona

Verificar que el usuario exista, que `set_password()` se haya usado y que `check_password()` valide correctamente.

5. Tabla no aparece en PostgreSQL

Ejecutar `flask db migrate` y luego `flask db upgrade`.

9. Archivos modificados en esta clase

Se modifican directamente

- `app/__init__.py`
- `app/config.py`
- `app/models.py`
- `app/auth/routes.py`
- `app/dashboard/routes.py`
- `.env`

Se generan

- `migrations/`

Se reutilizan de Clase 1

- `run.py`
 - `forms.py`
 - `base.html`
 - `login.html`
 - `styles.css`
-

10. Cierre docente

“En esta clase pasamos de una aplicación visual con login a un sistema conectado a una base de datos real. Ahora ya tenemos persistencia, modelos y migraciones. En la siguiente clase cargaremos el CSV de ventas y construiremos consultas agregadas para convertir datos en información comercial.”

11. Producto final de Clase 2

El paquete final de esta clase debe incluir:

- proyecto Flask funcional.
 - conexión activa a PostgreSQL.
 - tablas `usuarios` y `ventas` creadas.
 - usuario administrador cargado.
 - login validado contra base de datos.
 - dashboard mostrando al menos un dato desde PostgreSQL.
-

12. Recomendación docente

No avanzar a CSV ni dashboard todavía si los estudiantes no entienden:

- `db.Model`
- `db.Column`
- `db.session.add()`
- `db.session.commit()`
- `flask db migrate`
- `flask db upgrade` Estos conceptos son la base de todo el resto del proyecto.